KNU 강원대학교
KANGWON NATIONAL UNIVERSITY

R E S U L T R E P O R T

마이크로프로세서응용및실험 결과보고서_LCD활용



수 강 과 목	: 마이크로프로세서응용및실험-2분반
담 당 교 수	: 김도인 교수님
학 번	: 202210820
작 성 자	: 이동제
공 동 실 험 자	: 강민수(202214221)
제 출 일	: 2024-05-16 목요일

<목 차>

1. 실험목적 ····· 2

2. 실험원리 ····· 2

3. 실험기구 및 재료 ····· 3

4. 실험방법 ····· 3-9

5. 측정값 ····· 10

6. 실험결과 ····· 11

7. 결과에 대한 논의 ····· 11-12

8. 결론 ····· 12

9. 참고문헌 및 출처 ····· 12

1. 실험목적

LCD 화면에 원하는 위치에 원하는 문구를 띄우며, LCD의 사용 방법을 익힌다.

2. 실험원리

LCD 모듈은 V_{SS}, V_{DD}, V_L, RS, R/W, E, DB0~7, BLA, BLK 총 16개의 핀을 가지고 있으며, 각 기능들을 갖고 있습니다.
이러한 LCD 모듈을 제어하는 2개의 8비트 레지스터 IR(Instruction Reghister)과 DR(Data Register)를 가지고 실험합니다.

IR	LCD 모듈 제어에 관련된 정보가 저장됨 (커서 시프트, 화면 클리어, 글자 on/off 등) DD RAM의 어드레스를 저장함
DR	DD RAM에 Read/Write하는 문자를 저장함 (DR에 데이터를 써넣으면 자동으로 DD RAM에 저장됨)

표 1) IR과 DR

우리가 사용할 ATmega128의 LCD는 16 x 2개의 문자를 입력할 수 있고, 그 문자는 5 x 8 dots의 크기를 갖습니다.
그리고 문자나 문자열을 원하는 위치에 작성하기 위해서 DD RAM의 어드레스를 정확히 숙지해야 합니다.

DD RAM은 최대 80개의 문자를 저장할 수 있으며, 80개를 초과하여 작성하면 처음 주소로 돌아가서 작성합니다.
여기서 한 문자는 8비트의 ASCII code로, 이 문자가 80개까지 존재할 수 있습니다.
DD RAM의 어드레스는 첫 번째 행 (00H~27H)과 두 번째 행 (40H~67H)으로 나뉩니다.

RS와 R/W는 기본적으로 DD RAM으로 데이터를 써넣을 것인지, 읽어 올 것인지를 결정할 수 있습니다. 그리고 데이터를 쓰기로 했을 때 DB0~DB7에 해당하는 비트에 데이터를 입력하여서 커서를 홈 위치로 이동하거나, 데이터를 작성하거나 읽을 때 커서 진행 방향을 결정하는 등 LCD 모듈을 제어할 수 있습니다.

입력할 문자는 ASCII 코드를 기반으로 작동하기 때문에,
ASCII Table을 보면서 어떤 문자를 입력할 때 어떤 16진수(0x??)가 쓰이는지 알아야 합니다.

이러한 실험 원리들을 가지고 LCD 모듈 핀에 상응하는 비트에 데이터를 입력해 LCD 화면에 원하는 글자를 출력하는 테스트를 진행합니다.

3. 실험기구 및 재료

ATmega128 마이크로 컨트롤러	1개
컴퓨터	1개
ISP 케이블	1개

표 2) 실험기구 및 재료

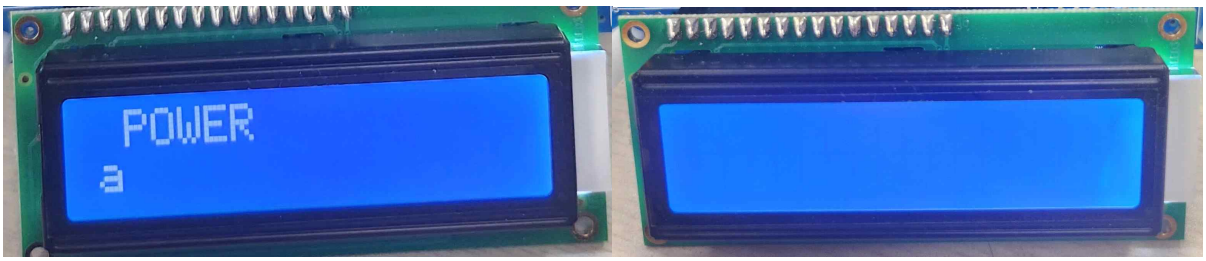
4. 실험방법

LCD를 동작시키는 테스트를 진행하기 위해 AVR Studio에서 코드를 작성합니다.
이번 실습교재8의 예제를 통해, 코드를 입력 후 ATmega128 동작 시 첫 번째 행에는 POWER라는 대문자 문자열이 출력되었고, 두 번째 행에 a라는 문자 하나가 출력되며 정상 작동하는 것을 확인했습니다.

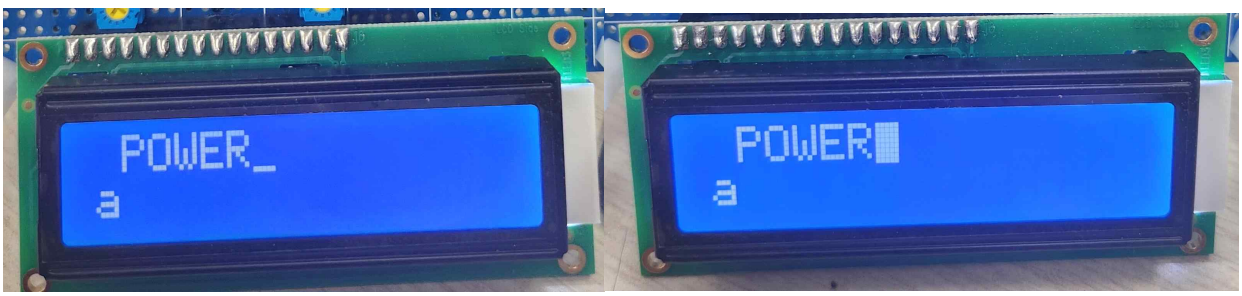
후에 구술할 실험방법은 추가로 진행된 실습에 대해 작성하겠습니다.

1. Display on/off control(disp_on, cursor_off, blink_off)과 Entry mode set(Increment mode, Entire shift_off)을 조절하여 각 동작 모드에 대한 결과를 확인해 보시오.

1-1. Display on/off control (0000 1'D,C,B')에 대한 동작 모드 변경 및 결과
(D - Display on/off, C - Cursor on/off, B - Blink on/off)



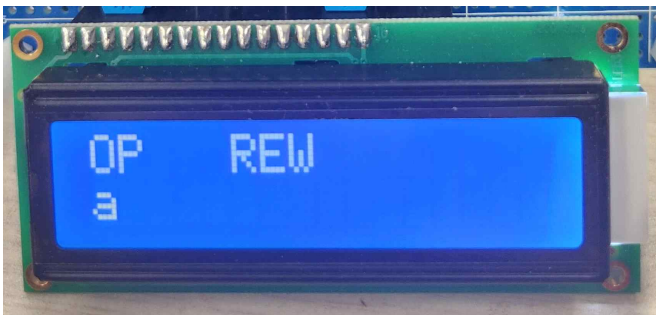
Display on / 0x0c (0000 1100) 입력 시 :
디스플레이가 켜집니다. 커서와 블링크는 off 되었기 때문에 보이지 않습니다.
Display off / 0x08 or 0x0b (0000 1000or1011) 입력 시 :
디스플레이가 꺼집니다. 때문에 커서와 블링크를 on 하여도 보이지 않습니다.



Cursor on / 0x0e (0000 1110) 입력 시 : 문자열 옆에 작대기 모양의 커서가 켜집니다.
Blink on / 0x0d (0000 1101) 입력 시 : 문자열 옆에 네모 박스 모양의 블링크가 켜집니다.

1-2. Entry mod set (0000 01'I/D,SH')
(I/D : 커서의 진행방향, SH : 표시된 문자 시프트)

Increment mode, Entire shift off 0000 0110 입력 시 :
커서가 오른쪽으로 진행되며, 문자 시프트가 꺼졌습니다.
Decrement mode, Entire shift off 0000 0100 입력 시 :
커서가 왼쪽으로 진행되며, 문자 시프트가 꺼졌습니다.



Increment mode, Entire shift on 0000 0111 입력 시 :
오른쪽으로 커서가 진행되며, 문자 시프트가 켜집니다.
문자가 시프트되어 다른 형태로 출력되었습니다.

2. 외부 인터럽트를 사용한 LCD 제어, 문자열 출력 및 문자열 쉬프트를 동작하는 프로그램 작성하기

2-1. 외부 인터럽트를 사용한 LCD 제어 (KNU IT 0123456789, power elec)

```
#define F_CPU 16000000L
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
```

기본값으로 CPU의 클럭 주파수를 16MHz로 설정시키고, 입출력 포트 및 레지스터에 대한 정의를 시키고, 지연(Delay) 함수와 인터럽트(interrupt) 함수를 사용하기 위해서 AVR 라이브러리 속 헤더 파일을 포함하는 코드를 입력합니다.

```
int a = 0;
```

LCD 상태를 제어하기 위해 a=0이라는 변수를 가정합니다.

```
void lcd_inst(unsigned char value) // 명령어 입력
{
        PORTA &= ~0x02;
        _delay_ms(1);
        PORTA |= 0x08;
        _delay_ms(1);
        PORTC=value;
        _delay_ms(1);
        PORTA &= 0x00;
        _delay_ms(1);
        }
```

lcd_inst는 LCD에 명령어를 전달하기 위한 함수로, PORTA에 비트 AND 연산자(&=)를 사용하여 RS 값을 0으로 만들어줍니다. 그리고 비트 OR 연산자(|=)를 사용하여 Enable을 1로 만들어 켜줍니다.
PORTC = value; 라는 명령어를 통해 인스트럭션 값을 PORTC에 출력하라고 설정합니다.
출력 후에는 0x00을 통해서 E와 RS를 모두 0으로 만듭니다.

그리고 LCD가 안정적으로 동작할 수 있도록 명령어와 데이터 사이에 텀을 만들어 주기 위해
중간마다 지연(delay) 함수를 넣어줍니다.

```
void lcd_init() // LCD 초기화
{
        _delay_ms(50);
        lcd_inst(0x38);
        _delay_ms(1);
        lcd_inst(0b00001100);
        _delay_ms(1);
        lcd_inst(0x01);
        _delay_ms(1);
        lcd_inst(0b00000110);
        _delay_ms(1);
}
```

lcd_init는 LCD의 사용 환경을 설정하는 함수입니다. 0x38(0011 1000)는 한 글자당 8-bit으로 사용할 것이며, 글자의 크기를 5 x 8 dots으로, 글자가 나타날 줄을 2개의 줄에서 표현하겠다고 설정합니다. 그리고 lcd_inst(0b00001100)을 통해 디스플레이를 on시키고 커서와 블링크를 off시켰습니다. 마지막은 커서의 진행방향과 표시된 문자를 시프트할 것인지 정하는 것인데, 진행 방향을 오른쪽으로 하고, 문자 시프트는 꺼두었습니다.

```
void lcd_ch(unsigned char ch) // LCD 데이터 입력
{
        PORTA |= 0x02;
        _delay_ms(1);
        PORTA |= 0x08;
        _delay_ms(1);
        PORTC=ch;
        _delay_ms(1);
        PORTA &= 0x00;
        _delay_ms(1);
}
```

lcd_ch는 데이터를 입력하는 함수입니다. 어느 데이터 레지스터를 선택하여 사용할 것인지 정합니다.
0x02로 PA1에 1이라는 값을 입력해 RS을 on하는 데이터 레지스터를 선택하고, 0x08로 PA3에 1를 대입하여 Enable을 작동시킵니다. 그리고 PORTC에 문자 값을 출력한다고 입력합니다.
출력 후에는 0x00을 통해서 E와 RS를 모두 0으로 만듭니다.

```
void lcd_out(unsigned char add, unsigned char cc) // LCD 주소 지정 및 문자 출력
{
        lcd_inst(add | 0x80);
        _delay_ms(1);
        lcd_ch(cc);
}
```

lcd_out은 번지를 지정해주기 위한 함수인데 add(주소)에 0x80 (0b00001000)를 입력하여 가장 최상위 비트 1로 설정합니다. 이제 LCD는 지정된 위치에 문자를 출력합니다.

```
void lcd_print(char line, char *text) // LCD 문자열 출력
{
        lcd_inst(line | 0x80);
        while(*text)

        lcd_ch(*text++);
}
```

lcd_print는 문자가 아닌 문자가 모여있는 문자열을 출력하기 위한 함수입니다. 똑같이 line(줄)에 0x80을 입력하여 최상위 비트 1로 설정되고, 지정된 위치에서 문자열이 출력됩니다.

```
ISR(INT0_vect) // 인터럽트 함수
{
        a = 1;
}

ISR(INT1_vect)
{
        a = 3;
}
```

ISR는 저번 실습때 사용하였던 인터럽트 함수로 외부 인터럽트 INT 0 발생 시 a를 1로 설정하고, INT 1 발생 시 a를 3으로 설정하게 하여 메인 함수 내에 대응되는 IF문의 동작을 할 수 있도록 합니다.

```
int main(void) // 메인 함수
{
        DDRC=0xff;
        DDRA=0x0A;
        lcd_init();
        _delay_ms(20);

        SREG |= 0x80;
        EIMSK=0x03;
        EICRA=0x0F;

        lcd_print(0x00, "KNU");
```

메인함수를 시작시킵니다. DDRC와 DDRA를 통해 PORTC와 PA1, PA3를 출력으로 설정합니다.
그리고 다시 lcd_init 함수를 통해 LCD를 초기화시킵니다.
SPEG와 EIMSK로 인터럽트를 허용하고 그중에 INT0과 INT1의 인터럽트를 활성화시킵니다.
EICRA는 인터럽트가 어느 지점에서 발생하여 동작할건지를 설정하는데 0x0f를 대입해 상승 엣지에서 동작하도록 설정합니다.
초기값으로 KNU라는 문자열이 항시 출력되고 있게 만듭니다.

```
        while(1){

                if(a==1){
                lcd_inst(0x01);
                _delay_ms(100);
                a=2;
                }
```

while문으로 3개의 IF문 내에서 동작을 반복하도록 합니다.
a가 1일 때 동작하는 IF문으로, a가 1이라는 변수로 지정되었을 때 LCD를 초기화하고 a에 2를 대입하여 a == 2일 때 동작으로 넘어가게 설정합니다.

```
                if(a==2){
                lcd_print(0x00, "KNU");
                lcd_print(0x40, "IT");
                }
```

a가 2일 때 동작하는 IF문으로 시작 지점을 0x00, 0x40으로 설정해주어
각각 첫 번째 행에는 'KNU' 두 번째 행에는 'IT'이라는 두 개의 문자열을 출력하도록 하였습니다.

```
                if(a==3){
                lcd_inst(0x01);
                _delay_ms(100);
                a = 4;
                }
```

그리고 a가 3일 때 동작하는 IF문은 a가 3이라는 변수로 지정되었을 때 LCD를 초기화하고 a에 4를 대입하여 a == 4일 때 동작으로 넘어가게 설정합니다.

```
                if(a==4){
                lcd_print(0x00, "0123456789");
                lcd_print(0x40, "power elec");
                }


        }
}
```

a가 4일 때 동작하는 IF문은 시작 지점을 0x00(첫 번째 행), 0x40(두 번째 행)으로 설정해서
각각 첫 번째 행에는 '0123456789' 두 번째 행에는 'power elec'이라는 문자열을 출력하도록 하였습니다.

그리고 함수를 마치기 위해 마지막 중괄호로 닫아줍니다.

2-2. 문자열 출력 및 문자열 쉬프트 (POWER 왼쪽 오른쪽 쉬프트)

```
#define F_CPU 16000000L
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
```

기본값으로 CPU의 클럭 주파수를 16MHz로 설정시키고, 입출력 포트 및 레지스터에 대한 정의를 시키고, 지연(Delay) 함수와 인터럽트(interrupt) 함수를 사용하기 위해서 AVR 라이브러리 속 헤더 파일을 포함하는 코드를 입력합니다.

```
int a = 0;
int erase = 0;
```

LCD 상태를 제어하기 위해 a=0이라는 변수를 가정합니다. 그리고 문자가 시프트되면서 원래의 문자 자리에 문자를 남기지 않고 입력되기 위해 erase 변수를 쓴다고 가정합니다. (초기값 = 0)

```
void lcd_inst(unsigned char value) // 명령어 입력
{
        PORTA &= ~0x02;
        _delay_ms(1);
```

```
        PORTA |= 0x08;
        _delay_ms(1);
        PORTC=value;
        _delay_ms(1);
        PORTA &= 0x00;
        _delay_ms(1);
}

void lcd_init() // LCD 초기화
{
        _delay_ms(50);
        lcd_inst(0x38);
        _delay_ms(1);
        lcd_inst(0b00001100);
        _delay_ms(1);
        lcd_inst(0x01);
        _delay_ms(1);
        lcd_inst(0b00000110);
        _delay_ms(1);
}

void lcd_ch(unsigned char ch) // LCD 문자 출력
{
        PORTA |= 0x02;
        _delay_ms(1);
        PORTA |= 0x08;
        _delay_ms(1);
        PORTC=ch;
        _delay_ms(1);
        PORTA &= 0x00;
        _delay_ms(1);
}

void lcd_out(unsigned char add, unsigned char cc) // LCD 주소 지정 및 문자 출력
{
        lcd_inst(add | 0x80);
        _delay_ms(1);
        lcd_ch(cc);
}

void lcd_print(char line, char *text) // LCD 문자열 출력
{
        lcd_inst(line | 0x80);
        while(*text)

        lcd_ch(*text++);
}
```

↑ void 함수들은 위에서 사용한 함수들과 똑같이 사용되기 때문에 반복되는 설명문은 생략했습니다.

```
ISR(INT0_vect)
{
        if(a>-5){
                a = a - 1;
                erase = 1;
        }
```

```
}

ISR(INT1_vect)
{
        if(a<6){
                a = a + 1;
                erase = 1;
        }
}
```

문자열이 LCD 화면 안에서만 동작할 수 있도록 하기 위해서 a가 -5보다 클 때만 동작하는 IF문을 사용합니다. INT0이 발생할 때마다 a에 a-1이란 값을 대입하여 왼쪽으로 시프트되게 만듭니다. 한 번 대입될 때마다 erase에 1을 대입하여 원래 입력되었던 문자열이 지워지게 합니다. 최대값인 a==-4까지 동작하면 더 이상 동작하지 않게 됩니다.

비슷하게 오른쪽으로도 나가지 않게 하기위해 a가 6보다 작을때만 동작하는 IF문을 사용합니다.
그리고 INT1이 한 번 발생할 때마다 a에 a+1 값을 대입하여 오른쪽으로 시프트되게 만듭니다. 마찬가지로 한 번 대입될 때마다 erase에 값 1을 대입하여 본래 문자열이 지워지고 이동된 값만 남도록 합니다.
최대값인 a==5까지 동작하면 더 이상 동작하지 않게 됩니다.

```
int main(void)
{
        DDRC=0xff;
        DDRA=0x0A;
        lcd_init();
        _delay_ms(20);

        SREG |= 0x80;
        EIMSK=0x03;
        EICRA=0x0F;

        while(1){
                if(erase==1){
                        erase = 0;
                        lcd_inst(0x01);         //DB0에 1을 넣어서 LCD 표시 클리어, 커서 홈위치
                        _delay_ms(100);
                }

        lcd_print(0x45 + a, "POWER");
        }

}
```

while문을 통해 메인 함수 내 IF문과 lcd_print 함수를 반복 동작하게끔 합니다.
erase 변수에 1이 입력되었을 때 동작하는 IF문은 erase를 0으로 만들어 반복 동작하지 않게끔 바꿔준 후 LCD에 써져있는 문자열을 초기화하고 커서를 홈위치로 설정합니다.

그리고 lcd_print 함수를 통해 0x45 + a에 해당되는 위치부터 'POWER'라는 대문자 문자열을 항시 출력하고 있도록 만듭니다. + a 때문에 a의 값이 달라질 때마다 문자열의 위치도 옮겨지게 됩니다.

5. 측정값

초기상태를 지정해준 후 INT0과 INT1 발생 시 다른 LCD 문자(혹은 문자열)을 출력하도록 만들었다.

외부 인터럽트를 사용한 LCD 제어

초기 상태	‘KNU’라는 문자열이 첫 번째 행에 출력됨.
INT 0	첫 번재 행에는 ‘KNU’, 두 번째 행에는 ‘IT’이라는 문자열이 출력됨.
INT 1	첫 번재 행에는 ‘0123456789’라는 문자열과, 두 번째 행에는 ‘power elec’이라는 소문자 알파벳 문자열이 출력됨.

표 3) 외부 인터럽트를 사용한 LCD 제어 코딩 테스트 측정값

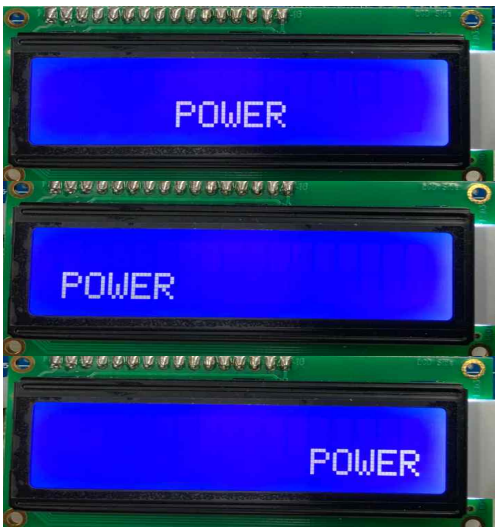
문자열 출력 및 문자열 쉬프트

초기 상태	두 번째 행에 ‘POWER’라는 문자열이 출력됨.
INT 0	INT 0이 발생할 때마다 POWER 문자열이 왼쪽으로 한 칸씩 시프트됨. LCD화면 왼쪽 끝에 도달하면 더 이상 움직이지 않음.
INT 1	INT 1이 발생할 때마다 POWER 문자열이 오른쪽으로 한 칸씩 시프트됨. LCD화면 오른쪽 끝에 도달하면 더 이상 움직이지 않음.

표 4) 문자열 출력 및 문자열 쉬프트 코딩 테스트 측정값

6. 실험결과





작성한 코드가 초기 상태에 LCD를 잘 출력하는지, INT0과 INT1 발생 시 다른 LCD 출력을 잘 동작하는지 테스트했다.

첫 번째 테스트, KNU 문자열을 출력하고 IT 문자열 출력까지는 순조로웠다. 하지만 0123456789와 power elec을 출력할 때 원래 있던 KNU와 IT 문자열이 지워지지 않아 겹쳐서 깜빡깜빡하는 동작을 반복하였다.

두 번째 테스트, 첫 번째 문제점을 해결하기 위해 INT0과 INT1이 발생 시 바로 동작하게끔 하지 않고 lcd_inst함수를 0x01로 설정해 본래 LCD에 입력된 값들을 클리어한 후 동작을 하게끔 코드를 작성했다. 문자열이 겹치지 않고 정상적으로 출력되었다.

세 번째 테스트, POWER를 왼쪽으로 시프트시키나 오른쪽으로 시프트시키나 LCD 화면 밖을 나가서 저 멀리 사라졌다.

마지막 테스트, POWER 문자열이 나가지 않도록 특정 값에 진입하면 더 이상 인터럽트를 발생시키지 않도록 ISR함수 안에 IF문을 넣고 경계조건을 설정해주었더니 더 이상 LCD 화면 밖으로 나가지 않으며 정상적으로 시프트 동작을 수행하였다.

7. 결과에 대한 논의

아쉬웠던 점은 새로운 문자열이 입력되면 자동으로 전에 있던 문자열이 지워질 줄 알고 작성했던 점이다. 이번 실습을 통해 새로운 문자열을 입력해주기 위해선 LCD를 한 번 클리어하고 작성해야 한다는 것을 배우게 되었다.
그리고 영어 타자가 많이 느려서 실습 시간에 코드를 입력하는 과정이 오래 걸리는 것도 많이 아쉬웠다. 실습 시간 전에 예제 코드를 미리 작성하는 준비를 하거나, 평소에 코드를 많이 작성하면서 영어 자판 위치에 익숙해지는 게 좋을 거 같다.

좋았던 점은 &= 와 |= 같은 AND 연산자, OR 연산자도 어떻게 동작하는지 알 수 있었고, LCD에 문자와 문자열을 첫 번째 행과 두 번째 행에 나누어서 출력할 수 있다는 것이 놀라웠다. 그리고 특히 저번에 사용하였던 ISR 함수 속 쉬프트 연산자를

이번엔 IF문을 사용하여 특정 값에 도달하면 사용하지 않도록 하는 방법을 알게 되어서 제일 기억에 남았다.
무작정 무한 반복을 하는게 아니라, 특정 값에 도달하면 동작을 안 하는게 참 신박하다고 느껴서 그런 것 같다.

확실히 LCD를 통해 직관적인 글자를 볼 수 있으니, LED를 통해 테스트했던 실습보다 비교적 가시성이 좋아 실습에 집중력이 높았던 것 같다. 다음엔 어떤 실습을 할지 궁금한 부분이다.

8. 결론

ATmega128 마이크로 컨트롤러 보드의 LCD 모듈 제어를 하는 레지스터와 그것에 해당하는 비트에 특정한 값을 입력해 LCD 화면에 원하는 위치에 원하는 문자를 출력하는 방법을 배웠다. 따라서 프로그램이 어떠한 동작을 할 때 이 동작은 무엇을 의미하는지 직관적으로 알려줄 수 있는 프로그램도 만들 수 있다는 것을 알게 되었다.

9. 참고문헌 및 출처

(교재) 김도인 교수님,「실습교재7」, 강원대학교, 전기전자공학과, (2~12p)
(교재) 김도인 교수님 [2024] 마프_Part 07_LCD_ver2, “, ”, (4~25p)
(사진) 이동제와 강민수, 실습 중 찍은 사진, 2024-05-09